Lógica em Software para Aplicação de Referência

Documento de viabilidade para validação dos resultados finais

1 Objetivo

A lógica não precisa reproduzir ciclos de clock nem a divisão do quadro em palavras de 64 bits.

Ela deve preservar a ordem temporal do PCAP e reproduzir as decisões funcionais que influenciam os resultados finais.

2 Escopo da primeira versão

Elemento	Definição
Entrada	Arquivo PCAP clássico lido diretamente, sem depender do Wireshark.
Protocolos	Ethernet II, IPv4, TCP e UDP.
Timestamp	80 bits: 48 bits para segundos absolutos e 32 bits para nanossegundos.
Fluxo	Chave bidirecional formada pelo protocolo e pelos dois endpoints ordenados.
Tabela	Tabela indexada por hash, com classificação MISS, HIT e COLLISION.
Exclusões iniciais	ARP, IPv6, VLAN, ICMP, fragmentação e opções IPv4.
Tratamento de erro	Registrar pacotes inválidos ou fora do escopo e continuar o processamento.

A aplicação de referência => usar os bytes e os timestamps do PCAP original.

3 Dados de entrada e modelo interno

3.1 Metadados de cada pacote

Campo	Conteúdo
packet_id	Número sequencial do pacote no PCAP.
timestamp_seconds	Parte de segundos do timestamp original.
timestamp_nanoseconds	Parte de nanossegundos, entre 0 e 999999999.
frame_length	Quantidade de bytes capturados no quadro.
ether_type	Tipo Ethernet; 0x0800 para IPv4.
src_ip e dst_ip	Endereços IPv4 de origem e destino.

Campo	Conteúdo
protocol	6 para TCP ou 17 para UDP.
src_port e dst_port	Portas de transporte.
tcp_flags	Flags TCP quando o protocolo for TCP.
payload_length	Quantidade de bytes após o cabeçalho de transporte.
valid e reason	Resultado do parser e motivo de rejeição, quando aplicável.

3.2 Chave do fluxo

A chave deve representar os dois sentidos de uma mesma comunicação. Para isso, os endpoints são ordenados antes da concatenação. Assim, um pacote enviado de A para B e a resposta de B para A consultam a mesma entrada da tabela.

```
endpoint_a = (src_ip, src_port)
endpoint_b = (dst_ip, dst_port)
low, high = sorted([endpoint_a, endpoint_b])
flow_key = (protocol, low, high)
```

A ordenação deve ser implementada de forma única e reutilizada pelo software e pelo RTL. A comparação da chave completa, e não somente do índice do hash, é necessária para detectar colisões.

3.3 Entrada da tabela de fluxos

Campo	Função
valid	Indica se a posição contém uma entrada ativa.
flow_key	Identifica o fluxo bidirecional.
flow_id	Identificador estável usado nos arquivos golden.
first_timestamp e last_timestamp	Definem o início, o fim e a duração observada.
packet_count e byte_count	Contadores totais do fluxo.
direction counters	Contadores separados para cada sentido.
length statistics	Mínimo, máximo, soma e média dos comprimentos.
interarrival statistics	Intervalos entre pacotes do mesmo fluxo.
tcp flag counters	Contagem de SYN, ACK, FIN, RST e PSH.
collision_count	Quantidade de conflitos observados no índice.

3.4 Conversão do timestamp

O timestamp do PCAP deve ser convertido sem usar float como representação intermediária. A parte inteira representa os segundos e a parte fracionária representa os nanossegundos. O valor de 80 bits é formado colocando os 48 bits de segundos acima dos 32 bits de nanossegundos.

```
value = Decimal(str(packet.time))
seconds = int(value)
nanoseconds = int((value - Decimal(seconds)) * Decimal(1000000000))
assert 0 <= nanoseconds <= 999999999
timestamp_80 = (seconds << 32) | nanoseconds
```

3.5 Funções da aplicação

Função	Entrada	Saída
read_pcap(path)	Caminho do PCAP	Registros em ordem temporal
parse_packet(record)	Bytes e timestamp	PacketMetadata
make_flow_key(metadata)	IPs, portas e protocolo	Chave canônica
hash_flow_key(key, config)	Chave e HashConfig	Hash, índice e tag
classify_and_update(entry, metadata)	Entrada e metadados	MISS, HIT ou COLLISION
finalize_entry(entry)	Entrada acumulada	Vetor de features
compare_golden(expected, observed)	Dois arquivos	Primeira divergência

4 Algoritmo de processamento

O processamento deve ocorrer exatamente na ordem dos registros do PCAP. Cada etapa produz dados observáveis, permitindo identificar se uma divergência surgiu no parser, na chave, no hash, na tabela ou na finalização.

```
flow_table = [empty_entry() for _ in range(TABLE_SIZE)]
events = []
invalid = []
for packet_id, record in enumerate(read_pcap(PCAP_FILE), start=1):
    metadata = parse_packet(record, packet_id)
    if not metadata.valid or metadata.out_of_scope:
        invalid.append(metadata)
        continue
    key = make_canonical_flow_key(metadata)
    hash_value = hash_unit(key)
    index, tag = reduce_hash(hash_value)
    entry = flow_table[index]
    if not entry.valid:
        status = MISS
```

```
        entry = create_entry(metadata, key, hash_value, tag)
        flow_table[index] = entry
    elif entry.flow_key == key:
        status = HIT
        update_entry(entry, metadata)
    else:
        status = COLLISION
        entry.collision_count += 1
        apply_collision_policy(entry, metadata, key)
    events.append(make_event(packet_id, status, index, hash_value, entry))
for entry in valid_entries(flow_table):
    features = finalize_entry(entry)
    write_flow_and_features(entry, features)
```

4.1 Tratamento de MISS

Ocorre quando o índice calculado aponta para uma posição vazia. A aplicação cria uma entrada, atribui um flow_id, copia os dados do primeiro pacote, inicializa os contadores e grava o evento.

4.2 Tratamento de HIT

Ocorre quando a posição está válida e a chave armazenada é igual à chave calculada. A aplicação atualiza os contadores, os tempos, o comprimento, a direção, os intervalos e as flags TCP.

4.3 Tratamento de COLLISION

Ocorre quando a posição está ocupada, mas a chave armazenada é diferente. A primeira versão deve registrar o conflito explicitamente, incrementar o contador e aplicar uma política definida no contrato. Não se deve sobrescrever silenciosamente uma entrada existente.

4.4 Finalização

Ao terminar o PCAP, a aplicação percorre todas as entradas válidas, calcula a duração e as estatísticas finais e gera um vetor de features por fluxo. O fim do arquivo é o evento de finalização da referência

4.5 Configuração que deve ser congelada

Parâmetro	O que deve ser igual no software e no RTL
TABLE_SIZE	Número de posições da flow table.
HASH_ALGORITHM	Algoritmo utilizado para transformar a chave em hash.
HASH_WIDTH	Quantidade de bits do hash produzido.
HASH_POLYNOMIAL e HASH_SEED	Polinômio, semente e inicialização, quando aplicáveis.
KEY_ENCODING	Ordem, largura e representação dos campos da chave.
INDEX_REDUCTION	Forma de obter o índice a partir do hash.

Parâmetro	O que deve ser igual no software e no RTL
COLLISION_POLICY	Ação quando o índice está ocupado por outra chave.
FEATURE_LAYOUT	Ordem, largura e sinal dos campos do vetor final.

5 Organização mínima do software

Arquivo	Responsabilidade
main.py	Coordena a execução e os caminhos dos arquivos.
pcap_reader.py	Lê o PCAP e preserva bytes, comprimento e timestamp.
packet_parser.py	Extrai e valida Ethernet II, IPv4, TCP e UDP.
flow_key.py	Ordena endpoints e constrói a chave bidirecional.
hash_unit.py	Calcula o hash determinístico, índice e tag.
flow_table.py	Consulta a tabela e executa MISS, HIT e COLLISION.
feature_extractor.py	Atualiza estatísticas e gera o vetor final.
comparator.py	Compara os arquivos golden com os registros produzidos pelo RTL.

6 Arquivos de saída para validação

Arquivo	Conteúdo
packet_metadata_golden.jsonl	Um registro por pacote aceito, com os metadados extraídos.
flow_events_golden.jsonl	Um registro por pacote aceito, com hash, índice, flow_id e status.
flows_golden.jsonl	Estado final de cada fluxo, com contadores e duração.
features_golden.jsonl	Vetor final de features usado como referência.
invalid_packets.txt	Pacotes inválidos ou fora do escopo e os motivos.
validation_summary.txt	Resumo de pacotes, fluxos, colisões, erros e comparações.

7 Critério de viabilidade

A aplicação será considerada viável quando executar o mesmo PCAP de forma repetível e produzir resultados que possam ser comparados campo a campo com o RTL.

- A quantidade e a ordem dos pacotes aceitos são iguais nos dois modelos.
- Os metadados do pacote coincidem: protocolo, IPs, portas, flags, comprimento e timestamp.
- A chave canônica, o hash, o índice e a tag coincidem.
- Cada pacote recebe o mesmo status MISS, HIT ou COLLISION e o mesmo flow_id.
- Os contadores, a duração e o vetor final de features coincidem.
- Diferenças são registradas com packet_id, campo esperado, campo obtido e etapa provável.

7.1 Resultado esperado para o PCAP atual

Para o conjunto atualmente gerado por gera_estimulos_mac.py, a aplicação deve reconhecer sete pacotes e dois fluxos bidirecionais. Os valores de hash e índice dependem da configuração congelada; a sequência de classificação abaixo deve permanecer igual quando não houver colisão.

Pacote	Tipo	Bytes	Fluxo	Status esperado
1	TCP SYN	54	F0	MISS
2	TCP SYN ACK	54	F0	HIT
3	TCP ACK	54	F0	HIT
4	HTTP request sobre TCP	122	F0	HIT
5	HTTP response sobre TCP	125	F0	HIT
6	UDP request	63	F1	MISS
7	UDP response	63	F1	HIT

Fluxo	Pacotes	Bytes	Característica
F0 TCP	5	409	SYN, SYN ACK, ACK e dados HTTP
F1 UDP	2	126	Requisição e resposta UDP
Total	7	535	69 palavras de 64 bits na validação do MAC

8 Casos mínimos para a primeira validação

Caso	Verificação

Caso	Verificação
TCP no mesmo fluxo	Vários pacotes nos dois sentidos devem produzir um MISS e HITs posteriores.
UDP no mesmo fluxo	Pacotes de requisição e resposta devem compartilhar a mesma chave.
Fluxos diferentes	Protocolos ou endpoints diferentes não podem compartilhar o flow_id.
Tamanhos diferentes	Comprimentos, payloads e estatísticas devem ser preservados.
Mais de uma transferência	O pacote deve continuar sendo tratado como um único registro funcional.
Colisão forçada	O contador e a política de colisão devem ser observáveis.
Pacote inválido	O pacote deve ser registrado e não interromper os demais casos.
Fora do escopo	ARP, IPv6, VLAN ou ICMP . Talvez descartar